



DEPARTMENT OF
COMPUTER SCIENCE

YAROSLAV HAYDUK

Bachelor in Computer Science and Engineering

CHALLENGES IN LEARNING AND TEACHING SOFTWARE MODELLING: A SOCIO-TECHNICAL GROUNDED THEORY APPROACH

MASTER IN INTEGRATED MASTER IN COMPUTER SCIENCE AND ENGINEERING

NOVA University Lisbon

Draft: January 19, 2026

CHALLENGES IN LEARNING AND TEACHING SOFTWARE MODELLING: A SOCIO-TECHNICAL GROUNDED THEORY APPROACH

YAROSLAV HAYDUK

Bachelor in Computer Science and Engineering

Supervisor: Miguel Goulão

Professor, NOVA University Lisbon

Co-supervisor: Grischa Liebel

Associate Professor, Reykjavik University

ABSTRACT

Regardless of the language in which the dissertation is written, usually there are at least two abstracts: one abstract in the same language as the main text, and another abstract in some other language.

The abstracts' order varies with the school. If your school has specific regulations concerning the abstracts' order, the NOVAthesis L^AT_EX (`novathesis`) (L^AT_EX) template will respect them. Otherwise, the default rule in the `novathesis` template is to have in first place the abstract in *the same language as main text*, and then the abstract in *the other language*. For example, if the dissertation is written in Portuguese, the abstracts' order will be first Portuguese and then English, followed by the main text in Portuguese. If the dissertation is written in English, the abstracts' order will be first English and then Portuguese, followed by the main text in English. However, this order can be customized by adding one of the following to the file `5_packages.tex`.

```
\ntsetup{abstractorder={<LANG_1>,...,<LANG_N>}}  
\ntsetup{abstractorder={<MAIN_LANG>={<LANG_1>,...,<LANG_N>}}}
```

For example, for a main document written in German with abstracts written in German, English and Italian (by this order) use:

```
\ntsetup{abstractorder={de={de,en,it}}}
```

Concerning its contents, the abstracts should not exceed one page and may answer the following questions (it is essential to adapt to the usual practices of your scientific area):

1. What is the problem?
2. Why is this problem interesting/challenging?
3. What is the proposed approach/solution/contribution?
4. What results (implications/consequences) from the solution?

Keywords: One keyword · Another keyword · Yet another keyword · One keyword more
· The last keyword

Sustainable Development Goals (SDG):



CONTENTS

Acronyms	vii
1 Introduction	1
2 Background	2
2.1 Software modelling & conceptual modelling	2
2.1.1 Purpose and role of modelling in software engineering	2
2.1.2 Conceptual modelling as a broader discipline	2
2.1.3 Modelling methods, languages, and artefacts	3
2.1.4 Implications for the scope of this thesis	3
2.2 Modelling education	4
2.2.1 Overview of conceptual modelling education	4
2.2.2 Bloom-based perspectives on modelling learning outcomes	4
2.2.3 Process and goal modelling education	5
2.2.4 Modelling, tools, and model-driven engineering in education	5
2.2.5 Competence-oriented and human-centred perspectives	6
2.2.6 Implications for this thesis	7
2.3 Grounded Theory	7
2.3.1 Overview and key characteristics	7
2.3.2 Grounded Theory in software engineering	8
2.3.3 Socio-Technical Grounded Theory	8
2.3.4 Implications for this thesis	9
2.4 Bloom’s Taxonomy and learning outcomes	9
2.4.1 Overview of Bloom’s revised taxonomy	9
2.4.2 Learning outcomes in computing and software engineering	10
2.4.3 Bloom-based formulations of modelling learning outcomes	10
2.4.4 Bloom, competencies, and this thesis	11
2.5 Human factors in modelling and SE education	11
2.5.1 Motivation, engagement, and the “value” of modelling	11

2.5.2	Inclusion, domain diversity, and participation barriers	12
2.5.3	Tools as mediators of learning experience	12
2.5.4	Authenticity, experiential learning, and social dynamics	13
2.5.5	Competence development and human-centred educational design	13
2.5.6	Implications for this thesis	13
3	Related Work	15
3.1	Empirical studies on learning/teaching specific modelling frameworks (iStar, UML, data models)	15
3.2	Broader SE education work on experience, competencies, experiential learning	15
3.3	Existing theories/frameworks (CaMeLOT, Domain Modelling in Bloom, etc.)	15
4	Methodology	16
4.1	Research design and rationale	16
4.2	Research questions	16
4.3	Study context and participants	16
4.4	Data collection	16
4.4.1	Instruments and pilot	16
4.4.2	Interviews	16
4.4.3	Surveys	16
4.4.4	Additional artefacts	16
4.5	Data analysis	16
4.5.1	Grounded Theory variant and socio-technical stance	16
4.5.2	Coding process	16
4.5.3	Theoretical sampling and saturation	16
4.5.4	Bloom’s taxonomy as a sensitising concept	16
4.6	Rigour and trustworthiness	16
4.7	Ethics and data management	16
4.8	Methodological limitations	16
	Bibliography	17

LIST OF FIGURES

LIST OF TABLES

ACRONYMS

novathesis NOVAthesis L^AT_EX (*p. i*)

INTRODUCTION

BACKGROUND

2.1 Software modelling & conceptual modelling

2.1.1 Purpose and role of modelling in software engineering

Software systems are complex socio-technical constructs that must satisfy functional and non-functional requirements while evolving under technical, organisational, and human constraints. Within this context, software modelling supports reasoning about software systems by providing abstractions that capture relevant aspects of a domain or system at an appropriate level of detail. Models can serve multiple roles: they may act as design blueprints, facilitate communication among stakeholders, and enable analysis and validation of properties before implementation [13, 5].

Although modelling is often introduced through widely adopted notations such as UML or BPMN, it should not be reduced to “drawing diagrams”. Instead, models can be understood as structured representations expressed in a modelling language, grounded in that language’s semantics, and created for a specific purpose and audience. From this perspective, modelling is not merely documentation; it is a way of constructing and manipulating knowledge about a domain or system [5].

2.1.2 Conceptual modelling as a broader discipline

It is useful to distinguish between software modelling as a practice within software engineering and conceptual modelling as a broader discipline concerned with representing, structuring, and reasoning about domains and systems across fields. Conceptual modelling is not limited to software development alone; it is also used in organisational analysis, enterprise and service design, data-intensive systems, and other domains where abstraction is needed to manage complexity and support understanding [13]. Reviews of conceptual modelling education underline its interdisciplinary nature and identify a broad family of modelling subtypes—including data, process, goal, and enterprise modelling—that often evolve in parallel, with limited pedagogical or theoretical integration across subfields [13, 3, 2].

This disciplinary perspective is particularly relevant for research on teaching and learning software modelling. When modelling is treated primarily as a supplementary topic within software engineering courses, students may not fully appreciate its significance and may develop oversimplified views of its purpose and applicability. In contrast, the conceptual modelling perspective frames modelling languages as knowledge schemas: they define what can be expressed, how meaning is structured, and what kinds of analysis or tooling become possible. Therefore, understanding modelling requires engaging with both notational elements and the underlying semantics, as well as the modelling goals and the intended audience of the produced models [5].

2.1.3 Modelling methods, languages, and artefacts

Several authors emphasise that a modelling method is more than a set of symbols or a notation. A modelling method can be understood as comprising a modelling language (including syntax, semantics, and notation), a modelling procedure (the process or steps to create models), and supporting mechanisms and algorithms such as analysis, simulation, or transformation capabilities [4, 5]. This holistic view clarifies why students may be able to reproduce diagrammatic symbols yet still struggle to apply modelling meaningfully: the procedural and analytical aspects of modelling are often less explicitly taught or assessed.

From this viewpoint, the value of a modelling language is not only in enabling representation, but also in supporting reasoning. When a language is treated as a structured schema, models become more than static diagrams: they can become queryable, analysable, and transformable artefacts that enable deeper engagement with a domain. This also motivates tool support that can provide feedback on syntactic correctness, highlight semantic issues, help manage complexity, or even automate certain modelling tasks [2, 13].

2.1.4 Implications for the scope of this thesis

This thesis adopts the view that software modelling is an instantiation of the broader discipline of conceptual modelling. It therefore treats modelling as a discipline involving conceptual understanding of modelling constructs and their semantics, procedural competence in constructing and refining models from problem descriptions, analytical judgement of model quality and fit-for-purpose, and socio-technical awareness of how modelling practices are shaped by tools, learning contexts, and stakeholder needs. This framing aligns with work on modelling learning outcomes and competencies, which emphasises not only notational knowledge but also procedural skills, evaluative abilities, and reflective understanding [2, 3, 4, 14]. This perspective motivates the subsequent background on modelling education, grounded theory, and learning outcomes, and provides a foundation for investigating modelling challenges from both student and instructor perspectives.

2.2 Modelling education

2.2.1 Overview of conceptual modelling education

Conceptual and software modelling are widely taught across computing and software-related programmes, but the way these topics are taught and integrated into curricula remains fragmented. Maslov et al. [13] map the conceptual modelling education literature and show that educational work is scattered across various modelling sub-communities (e.g., data modelling, process modelling, enterprise modelling, etc.), which often evolve in parallel with limited reuse of frameworks, terminology, and learning theories across subfields [13]. Within this landscape, modelling appears in diverse course contexts (from introductory programming courses to advanced software engineering), and courses emphasise different modelling families and purposes (e.g., data-centric, process-centric, goal-oriented, etc.).

The same studies also highlight that much of the current practice in modelling education is driven by instructor experience and preferences, local institutional constraints, and the availability of tools, rather than by systematically articulated educational frameworks or empirically validated design principles [13]. While there is a broad consensus on the importance of modelling skills for software professionals, there is less agreement on which specific *competencies* learners should acquire, how these competencies should be scaffolded, and how learning should be assessed in a reliable and educationally meaningful way.

2.2.2 Bloom-based perspectives on modelling learning outcomes

In response to the fragmented state of modelling education, several authors have proposed to adapt Bloom’s revised taxonomy to conceptual and domain modelling, with the goal of making learning outcomes more explicit and comparable across courses and contexts. At a general level, these works start from the cognitive levels of remembering, understanding, applying, analysing, evaluating, and creating [9], and specialise them to the kinds of activities involved in modelling, such as interpreting models, constructing models from textual descriptions, critiquing model quality, or designing new modelling methods.

Bogdanova and Snoeck analyse learning outcomes and assessment tasks for domain modelling through the lens of Bloom’s revised taxonomy, identifying that existing tasks tend to concentrate on understanding, analysing, and creating conceptual knowledge while giving less attention to evaluation, procedural knowledge, or metacognition [3]. Building on this analysis, CaMeLOT provides a three-dimensional framework that combines Bloom’s cognitive levels with a knowledge dimension (factual, conceptual, procedural, metacognitive) and specific content areas of conceptual data modelling (e.g., classes, relationships, methods, quality, and tool support) [2]. This framework is instantiated with detailed example learning outcomes and can be used to design or audit courses.

Bork extends this Bloom-based perspective to both conceptual modelling and meta-modeling, mapping knowledge and cognitive dimensions to all components of a modelling

method (language, semantics, notation, procedure, mechanisms and algorithms) [4]. An evaluation of a “Smart City” teaching case using this framework reveals that even rich, practice-oriented material may still underemphasise certain cognitive levels (e.g., evaluation) or knowledge types (e.g., metacognitive knowledge). These works collectively suggest that Bloom’s taxonomy can provide a useful lens for reflecting on and improving modelling education, by making learning objectives more explicit and balanced across different dimensions.

These frameworks are particularly useful for structuring interview and survey questions because they translate modelling competence into observable tasks.

2.2.3 Process and goal modelling education

Beyond data and structural modelling, there is growing interest in how students learn process and goal-oriented modelling. For business process modelling with BPMN, Maslov outlines a research agenda for developing empirically validated process-modelling education that integrates insights from conceptual modelling education, process model quality research, and learning sciences [18]. This agenda highlights the need for systematic mapping of existing work, empirical studies of novice modellers’ behaviour and typical errors, and the design and experimental evaluation of BPMN-specific teaching strategies and feedback mechanisms. This reflects a broader recognition that teaching process modelling effectively requires more than just introducing notation; it requires attention to cognitive demands, quality criteria, and the kinds of feedback needed to help learners improve.

Goal and actor-oriented modelling introduces additional layers of complexity, as it requires learners to reason about stakeholder intentions, rationales, and dependencies. The iStar learning study explicitly investigates the challenges students face when learning iStar, using a combination of Bloom-based task framing and grounded theory analysis of tutorials, modelling sessions, and interviews [19]. The authors report difficulties at multiple cognitive levels, including understanding the semantics of iStar constructs, applying the notation to real requirements, and analysing models to assess completeness and appropriateness. They derive a catalogue of strategies and tool-supported requirements (e.g., conceptual clarification, syntax checking, process guidance, templates, visualisation, and semi-automation modelling from text) that could help address these challenges [19].

These strands of work on BPMN and iStar education illustrate that different modelling families pose distinct learning challenges and may require specialised pedagogical approaches. They also reinforce the broader themes identified in conceptual modelling education: the need for scaffolding from text to model, support for managing complexity, and explicit attention to how learners evaluate and refine their models.

2.2.4 Modelling, tools, and model-driven engineering in education

Modelling in educational settings is often mediated by tools, ranging from simple drawing tools and web-based modellers to industrial-grade model-driven engineering (MDE)

environments. Tool choice can influence students' perceptions of modelling and their opportunities to practice particular skills. Liebel et al. conducted a multi-case study of model-driven software engineering courses that use different tools (e.g., Umple, Papyrus) and place different emphasis on model-driven architecture and code generation [10]. Their results indicate that students' perceptions of tools are highly context-dependent: the same tool may be perceived positively in one course setup and negatively in another, depending on project design, available support, and the roles that models play (e.g., for communication versus strict code generation). They also find that perceptions of UML as modelling language are more positive and robust when models are used for requirements and analysis, rather than only as precise inputs to code generators [10].

These findings align with broader discussions of human factors in model-driven engineering, which argue that educational tools should prioritise usability, cognitive support, constructive feedback, and alignment with learning goals over industrial feature completeness [17]. From an educational perspective, this suggests that tool selection and integration should be carefully considered to ensure they support the intended modelling competencies and do not inadvertently increase cognitive load or distract from learning objectives.

2.2.5 Competence-oriented and human-centred perspectives

Broader software engineering education research brings in complementary competence-oriented and human-centred perspectives that can inform modelling education. Sedelmaier and Landes propose the EVELIN research agenda for identifying and developing required competencies in software engineering, arguing that curricula should be grounded in explicit competence models rather than only topic lists [14]. They introduce a six-level classification system (remember, understand, explain, use, apply, develop) for technical competencies and outline an iterative, data-driven approach to relating teaching methods and structural conditions to competency development [14]. This approach resonates with the Bloom-based frameworks discussed earlier, but emphasises the need for empirical validation and alignment with real-world professional requirements.

Work on human factors in modelling and software engineering education further highlights the role of motivation, inclusion, and authenticity. For example, studies on domain diversity and motivation in modelling courses show that educators' assumptions about which domains are motivating (e.g., games, automation) do not always match students' preferences, and that students value domains that connect to their interests and everyday or socially relevant contexts [16]. Students also report that feeling excluded by unfamiliar or culturally distant domains, or by assignment setups that do not fit their backgrounds, can hinder their engagement and learning [16]. More generally, experiential education initiatives emphasise the value of working on real systems, real tasks, and with real mentors to bridge the gap between classroom modelling exercises and the complexities of practice [7].

Taken together, these competence-oriented and human-centred perspectives reinforce the idea that modelling education should be designed not only around notations and tools, but also around the competencies to be developed, the diversity of learners, and the authenticity and inclusiveness of modelling tasks and domains.

2.2.6 Implications for this thesis

The literature on modelling education, Bloom-based frameworks, process and goal modelling, tooling, competencies, and human factors provides several important implications for this thesis. First, it suggests that modelling learning outcomes can be systematically described in terms of cognitive and knowledge dimensions, and that existing courses may underemphasise evaluation and metacognitive aspects of modelling competence [3, 2, 4]. Second, it shows that different modelling families (e.g., data, process, goal) pose distinct learning challenges, and that empirical studies such as the iStar learning work can be used to derive grounded requirements for educational support [19, 18]. Third, it highlights that tools and model-driven approaches can both enable and hinder learning, depending on how they are integrated into course designs and on the kind of feedback and affordances they provide [10, 17]. Finally, competence-oriented and human-centred research indicates that modelling education must be sensitive to students' motivations, backgrounds, and learning environments, and that explicit competence models can support more systematic design and evaluation of teaching.

These insights inform the research questions and methodological choices of this thesis, which aims to investigate how students and teachers experience modelling, which challenges they perceive, and what kinds of support may foster the development of modelling competencies in realistic educational contexts.

2.3 Grounded Theory

2.3.1 Overview and key characteristics

Grounded Theory (GT) is a qualitative research methodology originally developed in sociology with the aim of generating theory that is systematically grounded in data rather than deduced from prior hypotheses. Instead of starting from a fixed theoretical framework, GT proceeds iteratively: data collection and analysis occur simultaneously, and emerging insights guide subsequent sampling and questioning. Core practices include *coding* (assigning conceptual labels to segments of data), *constant comparison* (comparing new data with existing codes and categories), and *theoretical sampling* (collecting new data to refine or challenge emerging concepts), and extensive *memoing* to capture analytic ideas [15, 1].

The outcome of a GT study is not a set of descriptive themes, but a more abstract, integrated explanation of how participants handle a central problem or concern. This explanation is typically organised around a *core category* that captures a dominant pattern

in the data and links other categories together. Evaluation criteria emphasise the *fit* of concepts to the data, the *workability* of the emerging theory in explaining the studied phenomenon, its *relevance* to practitioners and researchers, and its *modifiability* in light of new data, rather than statistical notions of validity [1].

Over time, GT has evolved into several variants/flavours, including *Classic GT* (Glaser and Strauss), *Straussian GT* (Strauss and Corbin), and *Constructivist GT* (Charmaz). While these variants share core principles, they differ in the degree to which they prescribe specific coding procedures, the role of prior theory, and the epistemological stance of the researcher. For software engineering research, this diversity offers flexibility but also introduces risks of inconsistent or superficial use if methodological choices are not made explicit [15].

2.3.2 Grounded Theory in software engineering

In software engineering (SE), GT has been used to study topics that are difficult to capture through controlled experiments or surveys, such as developer experience, team practices, decision-making processes, and socio-technical phenomena. A critical review by Stol et al. [15] finds that GT has become increasingly popular in SE research, but that many studies lack methodological rigour, adopt only parts of GT procedures, and show limited reflexivity regarding the researcher's role [15]. To address these issues, they offer a set of guidelines on when GT is appropriate (e.g., for under-theorised, practice-oriented problems), how to choose among GT variants, and how to report GT studies in a way that allows readers to assess their quality.

Adolph et al. provide a concrete example of applying classical (Glaserian) GT to study how software practitioners cope with the process of development, while also reflecting on the practical challenges of conducting GT research in SE contexts [1]. They describe in detail how they conducted open coding on interviews and field observations, developed concepts and categories through constant comparison, and iteratively refined their sampling strategy to explore interesting phenomena in more depth. Their work highlights the importance of staying close to the data, avoiding overly mechanical use of qualitative analysis tools, and systematically documenting analytic decisions through memos.

These contributions establish GT as a suitable methodology for SE when the research goal is to generate mid-level substantive theory about human and organisational aspects of software development, and they provide concrete guidance for designing and reporting such studies.

2.3.3 Socio-Technical Grounded Theory

Hoda et al. propose Socio-Technical Grounded Theory (STGT) as a specialisation of GT for SE that explicitly foregrounds the socio-technical nature of software development [6]. STGT retains the core principles of GT (iterative data collection and analysis, constant comparison,

theoretical sampling, memoing) but emphasises the need to attend simultaneously to social and technical elements and their interactions. This involves, for example, paying attention to roles, norms, tools, artefacts, and organisational structures, and analysing how these co-evolve over time.

STGT also articulates expectations for rigour in terms of credibility, transferability, dependability, and confirmability, and provides detailed guidance on reporting, including the description of research context, participants, data sources, coding procedures, and the evolution of categories and core concepts [6]. Therefore, STGT offers SE researchers a reporting template tailored to the realities of socio-technical systems.

2.3.4 Implications for this thesis

This thesis investigates how students and teachers experience software and conceptual modelling, and how they perceive challenges, learning processes, and support needs. These are socio-technical questions that involve individual cognition, educational practices, tools, and organisational constraints, and they are not yet well theorised in the software modelling education literature. Grounded Theory is therefore an appropriate methodological choice for this work.

2.4 Bloom's Taxonomy and learning outcomes

2.4.1 Overview of Bloom's revised taxonomy

The original Bloom's taxonomy was presented as a hierarchical classification of educational objectives, describing increasingly complex cognitive activities from simple recall to higher-order thinking. Krathwohl's revision restructures the taxonomy into a two-dimensional framework that distinguishes between a *knowledge dimension* (factual, conceptual, procedural, metacognitive) and a *cognitive process dimension* (remember, understand, apply, analyse, evaluate, create) [9]. This revised taxonomy encourages educators to formulate learning outcomes that are explicit both about the type of knowledge involved and the level of cognitive processing expected from learners.

In the revised framework, *factual knowledge* concerns basic elements that students must know, such as terminology and specific details; *conceptual knowledge* involves relationships among elements and larger structures; *procedural knowledge* relates to methods, techniques, and criteria for using procedures; and *metacognitive knowledge* refers to awareness of one's own cognition and learning strategies. The cognitive process dimension ranges from lower-order processes such as remembering and understanding, through applying and analysing, to higher-order processes of evaluating and creating [9]. The combination of these two dimensions supports a more fine-grained articulation of what learners should know and be able to do.

2.4.2 Learning outcomes in computing and software engineering

In computing and software engineering education, Bloom's revised taxonomy has been widely adopted as a tool for structuring curricula, designing assessments, and aligning teaching with intended learning outcomes. Typical applications include mapping course objectives to cognition levels, ensuring that programmes do not over-concentrate on lower-order outcomes, and checking that assessment tasks actually require the intended level of performance. However, generic formulations of the taxonomy can be difficult to apply directly in specialised domains such as software modelling, where domain-specific activities (e.g., interpreting a diagram, constructing a model from text, critiquing a modelling method) require more tailored descriptions.

This has led to several domain-specific adaptations of Bloom's taxonomy in areas such as domain modelling, conceptual data modelling, and metamodeling [3, 2, 4]. These adaptations retain the underlying cognitive-process levels while refining the knowledge categories and example tasks to reflect the particular demands of modelling work.

2.4.3 Bloom-based formulations of modelling learning outcomes

Several works in the modelling education literature explicitly build on Bloom's revised taxonomy to describe and analyse modelling-related learning outcomes. Bogdanova and Snoeck adapt the taxonomy to domain modelling by specifying what each cognitive level can mean in this context, such as remembering notation elements, understanding the meaning of relationships, applying modelling rules to construct diagrams, analysing models for completeness and consistency, evaluating alternative designs, and creating new models from complex descriptions [3]. Using this adapted taxonomy, they classify a large set of assessment tasks and show that existing practice tends to focus on understanding, analysing, and creating conceptual knowledge, with comparatively little emphasis on evaluation tasks, procedural knowledge, or metacognitive learning outcomes [3].

Extending this work, CaMeLOT proposes a three-dimensional framework for conceptual data modelling that explicitly combines Bloom's cognitive levels with the knowledge dimension and specific content areas such as classes, relationships, constraints, modelling methods, quality criteria, and tool usage [2]. The authors instantiate this framework with example learning outcomes, demonstrating how it can be used to design, review, or compare modelling courses. Their analysis also suggests that metacognitive outcomes (e.g., reflecting on modelling strategies, recognising one's own limitations) are rarely made explicit in current curricula [2].

Bork further generalises the Bloom-based perspective by developing separate yet related taxonomies for conceptual modelling and metamodeling [4]. He maps the knowledge and cognitive dimensions to the components of modelling methods (language, semantics, notation, procedure, mechanisms and algorithms) and uses this mapping to evaluate a Smart City teaching case. The evaluation reveals an imbalance: many activities target creation and application at higher cognitive levels, while explicit evaluate-level tasks and

metacognitive activities are largely absent [4]. This pattern is consistent with the findings of Bogdanova and Snoeck and suggests that modelling education often emphasises the ability to produce models but gives less structured attention to helping students judge model quality or reflect on their modelling practice.

These Bloom-based formulations show that learning outcomes for modelling can be described systematically across different levels and types of knowledge, and they provide concrete examples of how to manage this in educational settings. They also highlight recurring gaps, particularly in relation to evaluative and metacognitive aspects of modelling competence.

2.4.4 Bloom, competencies, and this thesis

For this thesis, Bloom’s revised taxonomy and its modelling-specific adaptations serve two main purposes. First, they offer a conceptual lens for thinking about modelling-related learning outcomes and competencies, distinguishing, for example, between understanding modelling constructs, applying modelling procedures, analysing and evaluating models, and reflecting on one’s own modelling practice. Second, the documented imbalances in current modelling education (e.g., underrepresentation of evaluation and metacognitive outcomes) motivate a closer examination of how students and teachers experience modelling and what kinds of support they perceive as needed [3, 2, 4, 14].

2.5 Human factors in modelling and SE education

2.5.1 Motivation, engagement, and the “value” of modelling

Software and conceptual modelling are often perceived by students as either highly valuable (e.g., for understanding complex systems and communicating design intent) or as burdensome “extra documentation” that competes with implementation time [16]. Such perceptions are not merely a matter of individual preference; they reflect a broader set of human factors that shape engagement, learning effort, and ultimately competence development. In modelling education, learners frequently report that motivation depends on whether modelling tasks feel authentic, whether their outputs matter beyond grading, and whether modelling is integrated into a meaningful workflow rather than treated as a purely academic exercise.

Research on modelling education explicitly highlights motivation as a key factor. A study on domain diversity, motivation, and inclusion in software modelling education shows that educators’ assumptions about which domains are motivating do not always match students’ preferences [16]. Students tend to value domains that connect to their interests, everyday experiences, or socially relevant contexts, and they report higher motivation when they have some agency (e.g., choice or personalisation of the domain) [16]. These results suggest that human factors in modelling education cannot be separated

from how modelling tasks are framed, which domains they use, and whether students can see a clear purpose for modelling beyond producing a diagram.

2.5.2 Inclusion, domain diversity, and participation barriers

Human factors also include issues of inclusion and participation. Even when modelling topics are formally accessible, examples and assignments can implicitly privilege specific backgrounds, interests, or prior experiences. Domain-diversity research indicates that certain commonly used modelling domains (e.g., video games or specific technical automation scenarios) may be engaging for some students but alienating for others [16]. Students may also experience barriers related to group-work structures, competitive gamification elements, or assumptions about familiarity with particular cultural or socioeconomic contexts. These factors can influence not only motivation but also opportunities for practice and feedback, thereby affecting learning outcomes.

From a course-design perspective, these findings motivate the careful selection of modelling domains and assignment setups, balancing instructors' preferences and curricular relevance with the diversity of student experiences and interests. Providing alternative domains, allowing domain choice, and ensuring that examples do not rely on narrow cultural references can be considered human-centred strategies to improve participation and reduce disengagement [16].

2.5.3 Tools as mediators of learning experience

In modelling education, tools play a critical role as mediators of the learning experience. They actively shape how students interact with modelling concepts, including how easily they can experiment, receive feedback, manage complexity, and recover from errors. Usability issues, steep learning curves, or misalignment between tool features and learning objectives can lead to frustration and negative attitudes toward modelling, even when the underlying notation is conceptually useful.

Liebel et al. examine model-driven software engineering education through a multi-case study and report that students' perceptions of modelling tools are highly context-dependent [10]. The same tool may be perceived as useful or cumbersome depending on how it is integrated into the course design, what the modelling artefacts are used for (e.g., communication vs. code generation), and the level of support provided. They also find that students tend to have more positive perceptions of UML when models are used for analysis and communication, even if tool-related issues reduce satisfaction in code-generation contexts [10]. This suggests that negative experiences are frequently tied to the socio-technical configuration of the course (tool, task, expectations) rather than to modelling as a concept.

2.5.4 Authenticity, experiential learning, and social dynamics

Human factors in SE education also encompass the extent to which learning activities resemble real-world practice. Experiential education initiatives emphasise that authenticity can enhance motivation, engagement, strengthen soft-skill development, and improve students' understanding of the complexities of software practice. Holmes et al. identify three "dimensions of experientialism" in a large, distributed capstone programme: working on real projects (existing open-source systems and codebases), performing real tasks (non-trivial contributions), and engaging with real mentors (experienced developers embedded in the project) [7]. Students report that this combination helps them foster communication skills, teamwork, time management, and practical problem-solving abilities, while also deepening their technical understanding in authentic contexts [7]. Although modelling courses may not always be structured as capstones, the experientialism perspective is still relevant.

2.5.5 Competence development and human-centred educational design

Competence-oriented research in SE education argues that curricula should be grounded in explicit competence models of what students should be able to do, and that teaching methods and structural conditions should be linked to competence outcomes rather than treated as ad hoc choices [14]. Sedelmaier and Landes propose the EVELIN research agenda for identifying and developing required competencies in software engineering, introducing a multi-level classification system for technical competencies intended to describe progression from basic familiarity to the ability to develop new solutions [14]. This approach emphasises iterative improvement through measuring competence levels, identifying influencing factors (structural conditions and process variables), and refining course designs accordingly [14].

Adopting such a competence-oriented and human-centred perspective in modelling education may require more than selecting an engaging domain or user-friendly tool; it may also involve designing learning activities and assessments that support the systematic progression across competencies, including conceptual understanding, procedural ability to create and refine models, evaluative judgement of model quality, and reflective awareness of modelling choices and limitations. The reviewed literature indicates evaluation and metacognitive competencies are often underemphasised in current modelling education [3, 2, 4], and human factors research provides a complementary explanation: learners may disengage when feedback is insufficient, tasks feel artificial, tools are frustrating, or domains do not resonate with their interests and experiences.

2.5.6 Implications for this thesis

The human factors literature motivates this thesis in several ways. First, it highlights that modelling education should be analysed not only in terms of notations and learning

outcomes, but also in terms of student and teacher experiences shaped by motivation, inclusion, feedback, and perceived authenticity [16, 7]. Second, modelling tools should be treated as socio-technical mediators that influence cognitive load, engagement, and attitudes toward modelling, making tool choice and integration important pedagogical determinants of learning experience [10, 17]. Third, competence-oriented perspectives suggest that empirical investigation should seek to connect perceived challenges and support needs to explicit competence development and course design choices [14].

RELATED WORK

- 3.1 Empirical studies on learning/teaching specific modelling frameworks (iStar, UML, data models)**
- 3.2 Broader SE education work on experience, competencies, experiential learning**
- 3.3 Existing theories/frameworks (CaMeLOT, Domain Modelling in Bloom, etc.)**

METHODOLOGY

4.1 Research design and rationale

4.2 Research questions

4.3 Study context and participants

4.4 Data collection

4.4.1 Instruments and pilot

4.4.2 Interviews

4.4.3 Surveys

4.4.4 Additional artefacts

4.5 Data analysis

4.5.1 Grounded Theory variant and socio-technical stance

4.5.2 Coding process

4.5.3 Theoretical sampling and saturation

4.5.4 Bloom's taxonomy as a sensitising concept

4.6 Rigour and trustworthiness

4.7 Ethics and data management

4.8 Methodological limitations

BIBLIOGRAPHY

- [1] S. Adolph, W. Hall, and P. Kruchten. “Using Grounded Theory to Study the Experience of Software Development”. In: *IEEE Software* 28.4 (2011), pp. 24–27 (cit. on pp. 7, 8).
- [2] D. Bogdanova and M. Snoeck. “CaMeLOT: An Educational Framework for Conceptual Data Modelling”. In: *TBD* (2020) (cit. on pp. 2–4, 7, 10, 11, 13).
- [3] D. Bogdanova and M. Snoeck. “Domain Modelling in Bloom: Deciphering How We Teach It”. In: *TBD* (2020) (cit. on pp. 2–4, 7, 10, 11, 13).
- [4] D. Bork. “A Framework for Teaching Conceptual Modeling and Metamodeling Based on Bloom’s Revised Taxonomy”. In: *TBD*. 2020 (cit. on pp. 3, 5, 7, 10, 11, 13).
- [5] R. A. Buchmann, D. Karagiannis, and H.-G. Fill. “Conceptual Modelling in Education: A Position Paper”. In: *TBD*. 2020 (cit. on pp. 2, 3).
- [6] R. Hoda, J. Noble, and S. Marshall. “Socio-Technical Grounded Theory for Software Engineering”. In: *IEEE Transactions on Software Engineering* 44.12 (2018), pp. 1208–1226 (cit. on pp. 8, 9).
- [7] R. Holmes, R. J. Walker, and G. C. Murphy. “Dimensions of Experientialism for Software Engineering Education”. In: *TBD* (2014) (cit. on pp. 6, 13, 14).
- [8] B. Kitchenham and S. L. Pfleeger. “Teaching Survey Research in Software Engineering”. In: *ACM SIGSOFT Software Engineering Notes* 33.6 (2008), pp. 1–10.
- [9] D. R. Krathwohl. “A Revision of Bloom’s Taxonomy: An Overview”. In: *Theory Into Practice* 41.4 (2002), pp. 212–218 (cit. on pp. 4, 9).
- [10] G. Liebel et al. “Model Driven Software Engineering in Education: A Multi-Case Study on Perception of Tools and UML”. In: *TBD* (2014) (cit. on pp. 6, 7, 12, 14).
- [11] J. Linaker, P. Runeson, and B. Regnell. “Challenges in Survey Research”. In: *Empirical Software Engineering* 20.4 (2015), pp. 924–962.
- [12] J. M. Lourenço. *The NOVAtesis L^AT_EX Template User’s Manual*. NOVA University Lisbon. 2021. URL: <https://github.com/joaomlourenco/novathesis/raw/main/template.pdf>.

- [13] A. Maslov, S. Poelmans, and N. Rosenthal. "Conceptual Modeling Education". In: *TBD* (2021) (cit. on pp. [2–4](#)).
- [14] Y. Sedelmaier and D. Landes. "A Research Agenda for Identifying and Developing Required Competencies in Software Engineering". In: *TBD* (2014) (cit. on pp. [3](#), [6](#), [11](#), [13](#), [14](#)).
- [15] K.-J. Stol, P. Ralph, and B. Fitzgerald. "Grounded Theory in Software Engineering Research: A Critical Review and Guidelines". In: *IEEE Transactions on Software Engineering* 42.12 (2016), pp. 1201–1222 (cit. on pp. [7](#), [8](#)).
- [16] TBD. "Domain Diversity, Motivation, and Inclusion in Software Modelling Education". In: *TBD*. 2020 (cit. on pp. [6](#), [11](#), [12](#), [14](#)).
- [17] TBD. "Human Factors in Model-Driven Engineering: Future Research Goals and Initiatives for MDE". In: *TBD*. 2020 (cit. on pp. [6](#), [7](#), [14](#)).
- [18] TBD. "Towards Empirically Validated Process Modelling Education Using a BPMN Formalism". In: *TBD*. 2020 (cit. on pp. [5](#), [7](#)).
- [19] TBD. "Understanding the Challenges and Requirements for Facilitating iStar Learning: An Empirical Study with iStar Learners". In: *TBD*. 2017 (cit. on pp. [5](#), [7](#)).

